Embedded Systems Laboratory Report

Digital Filtering of Sensor Data with CMSIS-DSP and CMSIS-RTOS2

Basic Problem and Option Problem 1

YU-CHEN, CHU
Student ID: R12942157

November 11, 2026

Source code: `https://github.com/oldpig123/embedded_system/tree/master/lab7`

Contents

1 Introduction **2**

2 Environment **2**

3 FIR Filters in CMSIS-DSP **2**

4 Basic Problem **3**
4.1 First check of the library . . . 3
4.2 Test with a known signal . . . 3
4.3 Filter design for the accelerometer . . . 3
4.4 Sampling the sensor . . . 4
4.5 Results . . . 5

5 Option Problem 1: Three Tasks with CMSIS-RTOS2 **6**
5.1 Design . . . 6
5.2 RTOS configuration . . . 6
5.3 Task 1: timer-triggered sensor reads . . . 7
5.4 Task 2: filter . . . 8
5.5 Task 3: Wi-Fi . . . 9
5.6 PC side . . . 9
5.7 Results . . . 10
5.8 Discussion . . . 11

6 Problems Encountered **11**

7 Conclusion **12**

1 Introduction

CMSIS-DSP is Arm's signal processing library for Cortex-M processors. It contains filters, transforms, statistics and matrix functions, with versions that use the DSP instructions and the floating-point unit (FPU) of the Cortex-M4.

The basic problem asks for a DSP program that processes data from a sensor on the STM32 IoT node. I chose the accelerometer and a finite impulse response (FIR) low-pass filter. Before I used real data, I tested the program with a known signal. This test is the `arm_fir_example` from the lecture: a 1 kHz plus 15 kHz input with a reference output that MATLAB computed.

Option problem 1 puts the same processing into three CMSIS-RTOS2 tasks. The first task reads the sensor when a timer triggers it. The second task filters the data. The third task sends the raw and the filtered data over Wi-Fi to a PC, which plots both.

2 Environment

Table 1: Hardware and software.

Board	B-L475E-IOT01A (STM32L475VG, Cortex-M4F, 80 MHz)
Sensor	LSM6DSL accelerometer on I2C2, read with the board support package (BSP)
Configuration	STM32CubeMX 6.18.1, STM32Cube FW_L4 V1.18.2 (HAL)
DSP library	CMSIS-DSP V1.6.0, prebuilt `libarm_cortexM4lf_math.a` (little endian, hardware FPU)
RTOS	FreeRTOS V10.3.1 with the CMSIS-RTOS2 interface (option problem)
Wi-Fi	Inventek ISM43362 module on SPI3, ES-WiFi driver from STM32CubeL4
Build	CMake, Ninja, `arm-none-eabi-gcc` 14.3.1, VS Code with the STM32Cube extension
Serial output	USART1 to the ST-LINK virtual COM port, 1 Mbaud, 8N1, `pyserial` miniterm
Host tools	Python 3.14 with NumPy, SciPy and Matplotlib, managed with `uv`

The assignment names STM32CubeIDE. I used the same CubeMX code generator with a CMake project in VS Code. The HAL code and the build flags are the same.

The system clock comes from the MSI oscillator through the PLL, which gives 80 MHz. I copied the CMSIS-DSP headers and the library from the STM32CubeL4 package into `lab7/ThirdParty/`. The BSP files and the ES-WiFi driver are in the same folder. CubeMX regenerates `Drivers/` and `Middlewares/`, so it does not touch `ThirdParty/`. The project is self-contained and builds on a different PC without changes.

The C library is newlib-nano (`-specs=nano.specs`). Its `printf()` does not support `%f` unless the linker pulls in the float code. The linker option `-u _printf_float` does this.

3 FIR Filters in CMSIS-DSP

An FIR filter with N taps computes each output sample from the last N input samples:

$$y[n] = \sum_{k=0}^{N-1} h[k]\, x[n-k]. \tag{1}$$

A filter with symmetric coefficients has a linear phase. All frequencies leave it with the same delay of $(N-1)/2$ samples, so the filter changes the shape of a signal only through the gain.

`arm_fir_f32()` processes one block of `blockSize` samples per call. The instance structure points to a state buffer of `numTaps` + `blockSize` − 1 samples. After each call, the function keeps the last `numTaps` − 1 inputs in this buffer. The next block starts with them. Thus a stream that the program cuts into blocks gives the same output as one long call. `arm_fir_init_f32()` sets the state to zero. CMSIS-DSP expects the coefficients in time-reversed order. The filters in this lab are symmetric, so the order makes no difference here. My design script reverses them anyway.

4 Basic Problem

4.1 First check of the library

The first program adds two one-element vectors with `arm_add_f32()`: $1.0 + 2.75$. The board printed `Result: 3.750000`. This proves that the header, the library and the float `printf()` work together.

4.2 Test with a known signal

I ported `arm_fir_example_f32.c` from STM32CubeL4. The test data in `arm_fir_data.c` has 320 samples of a 1 kHz and a 15 kHz sine at $f_s = 48$ kHz, and the MATLAB output of the same filter. The filter is `fir1(28, 6/24)`: 29 taps, Hamming window, cutoff 6 kHz. The program calls `arm_fir_f32()` for 10 blocks of 32 samples. Then `arm_snr_f32()` compares the result with the reference:

$$\mathrm{SNR} = 10 \log_{10} \frac{\sum_n \mathrm{ref}[n]^2}{\sum_n (\mathrm{ref}[n] - \mathrm{out}[n])^2} \ \mathrm{dB}. \tag{2}$$

The test passes if the SNR is 140 dB or more. Float32 has a 24-bit mantissa, so two correct results differ only in the last bits, which is about 140 dB below the signal.

```
Result: 3.750000
SNR: 142.585007
Status: Success
```

Listing 1: Serial output of the known-signal test.

A test that cannot fail proves nothing. Thus I changed the center tap from 0.2504960933 to 0.3 and ran the test again. The board printed `SNR: 25.157251` and `Status: Failure`. A hand calculation predicts this value. The wrong tap adds the error $e[n] = \Delta\, x[n - 14]$ with $\Delta = 0.0495$. The input has the power $1^2/2 + 0.5^2/2 = 0.625$, and the reference (the 1 kHz sine) has the power 0.5. So

$$\mathrm{SNR} \approx 10 \log_{10} \frac{0.5}{0.0495^2 \times 0.625} = 25.1 \ \mathrm{dB}. \tag{3}$$

The first 15 outputs were also identical to the correct run. The wrong tap multiplies $x[n - 14]$, and $x[0] = 0$, so the first output that changes is $y[15]$.

The board printed the input and the output of the correct run between `BEGIN FIR` and `END FIR`. The script `plot_fir.py` captures this block and draws Figure 1, in the same layout as the lecture slide. The 15 kHz component is gone from the output. The output starts at zero and needs about 14 samples to rise, because the state buffer starts with zeros. The output is also 14 samples late, which is the group delay $(29 - 1)/2$.

4.3 Filter design for the accelerometer

A hand on the board or a tap on the table makes short spikes in the acceleration. The slow motion of the board is below a few hertz. I chose a sample rate of 50 Hz and a cutoff of 5 Hz.

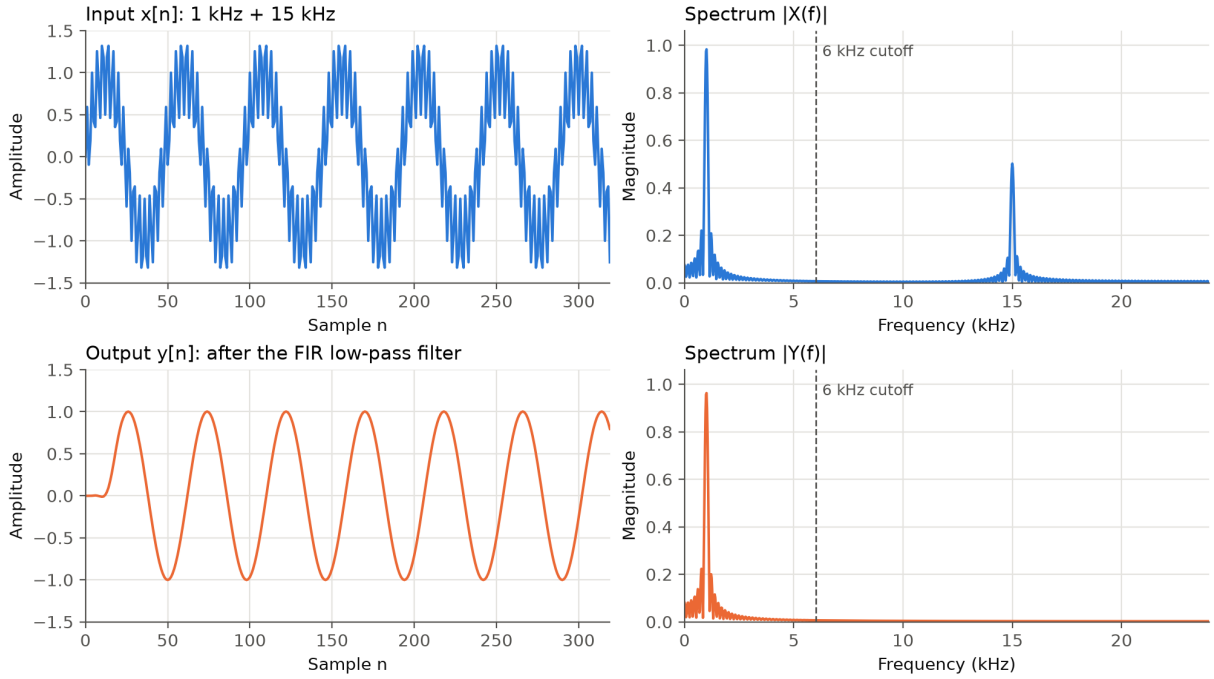


Figure 1: Known-signal test, measured on the board. Top: the input (1 kHz + 15 kHz). Bottom: the output of `arm_fir_f32()`.

The script `design_fir.py` designs the filter with `scipy.signal.firwin()` and a Hamming window. This is the same method as MATLAB `fir1()`. For the lecture filter, both give the same coefficients within 4×10^{-11}. The script prints the coefficients as a C array in time-reversed order, and it checks the gain:

Table 2: Accelerometer filter: 29 taps, $f_s = 50$ Hz, cutoff 5 Hz.

Frequency	Gain
0 Hz (gravity, slow tilt)	0.00 dB
5 Hz (cutoff)	−6.06 dB
10 Hz	−63.97 dB
Group delay	14 samples = 280 ms

For `fir1()` and `firwin()`, the cutoff is the −6 dB point, not the −3 dB point.

4.4 Sampling the sensor

`BSP_ACCELERO_Init()` sets the LSM6DSL to an output data rate of 52 Hz and ±2 g. `BSP_ACCELERO_AccGetXYZ()` gives the three axes in mg. The program uses the Z axis. When the board lies flat, Z is about 1000 mg (gravity).

The main loop reads the sensor every 20 ms. It keeps a deadline and adds 20 ms to the deadline, not to the current time. A late read thus does not shift all later reads. The comparison uses a signed difference, so it stays correct when `HAL_GetTick()` wraps after 49.7 days.

```
if ((int32_t)(HAL_GetTick() - next) >= 0) {
  next += 20;
  BSP_ACCELERO_AccGetXYZ(accel);
  accIn[k] = (float32_t)accel[2];
  k++;
  if (k == ACC_BLOCK_SIZE)
```

```
  {
    arm_fir_f32(&accFilter, accIn, accOut, ACC_BLOCK_SIZE);
    for ( int j = 0; j < ACC_BLOCK_SIZE; j++) {
      printf("%lu,%f,%f\n", n++, accIn[j], accOut[j]);
    }
    k = 0;
  }
}
```

Listing 2: 50 Hz sampling and block filtering (basic problem, `main.c`, user code block WHILE).

A block has 25 samples, which is 0.5 s. The filter has its own instance and its own state buffer, separate from the known-signal test. The program prints 25 lines of about 28 bytes after each block. At 115200 baud this takes about 60 ms, which is three sample periods. The deadline catches up after the print, but three reads then come back to back. At 1 Mbaud the print takes about 7 ms, so I raised USART1 to 1 Mbaud. This rate divides exactly from the 80 MHz clock of USART1 and from the 48 MHz clock of the ST-LINK.

4.5 Results

`plot_acc.py` records 10 s (500 samples) and draws Figure 2. I tapped the table twice during the recording.

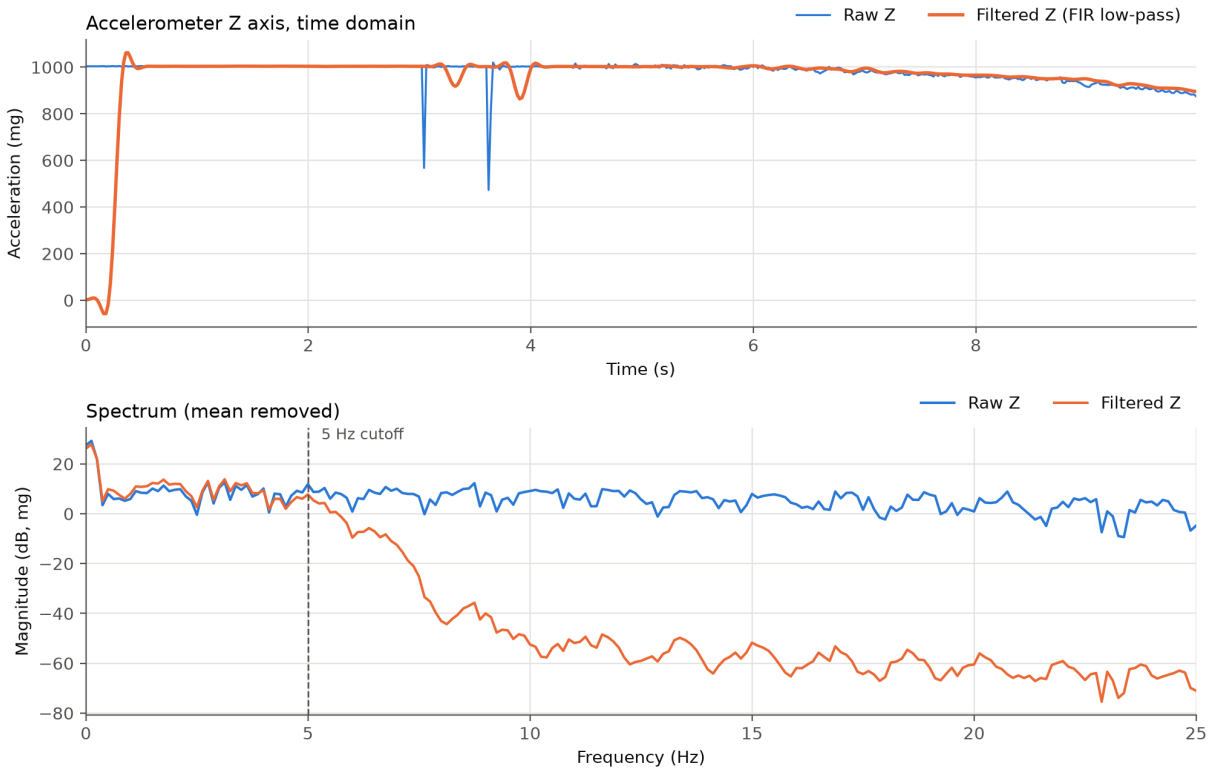


Figure 2: Basic problem: Z acceleration before and after the filter. Top: time domain. Bottom: amplitude spectrum with the mean removed.

- **Correctness.** I ran `scipy.signal.lfilter()` on the raw column with the same coefficients. The board output agrees within 2.3×10^{-4} mg on all 500 samples. The capture has 20 block boundaries, so this also proves that the state buffer carries the history from block to block.
- **Noise.** In a quiet second, the standard deviation of Z falls from 0.57 mg to 0.22 mg.

- **Spikes.** The taps reach 473 mg in the raw data. The filtered signal goes down only to 864 mg. The filter spreads each spike over about 0.5 s, and it rings a little after it.
- **Spectrum.** The two curves are equal below 5 Hz. Above about 8 Hz the filtered spectrum is 50 to 70 dB lower.
- **Start and delay.** The filtered curve starts at 0 and rises to 1000 mg in the first 0.56 s (28 samples), because the state starts at zero. Each filtered dip comes 280 ms after its spike.

5 Option Problem 1: Three Tasks with CMSIS-RTOS2

5.1 Design

The option problem uses a separate CubeMX project in `lab7/rtos/`. It starts from a copy of `lab7.ioc` and shares `lab7/ThirdParty/`. The basic project stays unchanged.

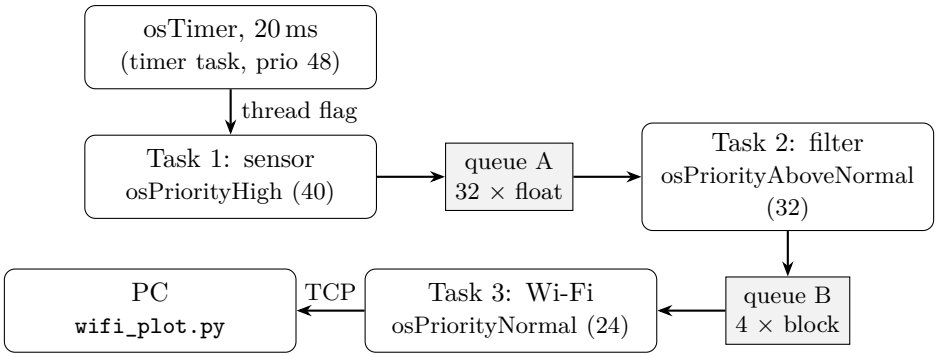


Figure 3: Tasks, timer and queues. A block holds the start index n_0, 25 raw samples and 25 filtered samples.

- **Task 1 (sensor)** has the highest priority, because it has the strictest timing. It reads one sample when the timer triggers it.
- **Task 2 (filter)** collects 25 samples, runs `arm_fir_f32()` and puts the raw and the filtered block into queue B.
- **Task 3 (Wi-Fi)** has the lowest priority. A Wi-Fi send takes a long and variable time, and no other task waits for it.

Each queue decouples a fast producer from a slower consumer. The producers put items with a timeout of 0. A full queue therefore never blocks a higher-priority task. Instead, the producer counts the lost item: `droppedSamples` for queue A and `droppedBlocks` for queue B. The Wi-Fi task sends both counters every 10 s.

5.2 RTOS configuration

An osTimer callback does not run in an interrupt. It runs in the FreeRTOS timer service task. CubeMX gives this task the priority 2, which is below all three of my tasks (24, 32 and 40). With that default, a busy Wi-Fi task delays the timer callback, and the highest-priority task wakes up late. With priority 48, the callback runs as soon as the timer expires. The callback only sets a flag, so this priority costs almost no CPU time.

Table 3: FreeRTOS settings in CubeMX.

Setting	Value and reason
Interface	CMSIS_V2
HAL timebase	TIM6. FreeRTOS uses SysTick for its own tick.
`TOTAL_HEAP_SIZE`	32768 bytes for three task stacks, two queues and the timer
Task stacks	512, 512 and 1024 words. `printf()` with `%f` needs more than the default 128 words.
`CHECK_FOR_STACK_OVERFLOW`	Option 2, so an overflow stops the program and does not corrupt memory
`USE_NEWLIB_REENTRANT`	Enabled. Each task gets its own C library state for `printf()`.
`TIMER_TASK_PRIORITY`	48 (osPriorityRealtime) instead of 2, see below

5.3 Task 1: timer-triggered sensor reads

```
void SampleTimerCallback(void *argument)
{
  osThreadFlagsSet(sensorTaskHandle, 0x01);
  sensorTickcounter++;
}

void StartSensorTask(void *argument)
{
  for(;;)
  {
    osThreadFlagsWait(0x01, osFlagsWaitAny, osWaitForever);
    int16_t xyz[3];
    BSP_ACCELERO_AccGetXYZ(xyz);
    sampleCount++;
    float32_t zValue = (float32_t)xyz[2];
    if (osMessageQueuePut(rawZQueueHandle, &zValue, 0, 0) != osOK) {
      droppedSamples++;
    }
  }
}
```

Listing 3: Timer callback and sensor task (`rtos/Core/Src/main.c`, comments shortened).

The I2C read happens in the task, not in the callback. Thread flags do not count. If the timer fires twice before the task waits again, the task wakes only once, and nothing reports the lost trigger. To detect this case, the callback counts the triggers and the task counts the samples. A test version printed both counters every 50 samples:

```
Sample Count: 50, Sensor Tick Counter: 50, Z Value: 1002, Elapsed Time: 1001 ms
Sample Count: 100, Sensor Tick Counter: 100, Z Value: 1001, Elapsed Time: 1000 ms
Sample Count: 150, Sensor Tick Counter: 150, Z Value: 1003, Elapsed Time: 1000 ms
...
Sample Count: 450, Sensor Tick Counter: 450, Z Value: 1003, Elapsed Time: 1000 ms
```

Listing 4: Test of task 1 (board flat on the table).

50 samples take 1000 ms, so the rate is exactly 50 Hz. The two counters are always equal, so no trigger was lost. The first line includes the start-up time.

The value goes into queue A as `float32_t`. The queue copies `sizeof(float32_t)` = 4 bytes from the given address. A pointer to the `int16_t` value would copy 2 bytes of the sample and 2 bytes of the stack after it.

5.4 Task 2: filter

```
osMessageQueueGet(rawZQueueHandle, &accIn[k], NULL, osWaitForever);
if (k == ACC_BLOCK_SIZE - 1) {
  arm_fir_f32(&accFilter, accIn, accOut, ACC_BLOCK_SIZE);
  AccDataBlock block;
  block.n0 = n;
  memcpy(block.raw, accIn, ACC_BLOCK_SIZE * sizeof(float32_t));
  memcpy(block.filt, accOut, ACC_BLOCK_SIZE * sizeof(float32_t));
  if (osMessageQueuePut(accDataQueueHandle, &block, 0, 0) != osOK) {
    printf("Failed to put block into queue\n");
    droppedBlocks++;
  }
  n += ACC_BLOCK_SIZE;
  k = 0;
} else {
  k++;
}
```

Listing 5: Filter task (comments and old test code removed).

The blocking `osMessageQueueGet()` sets the pace of the task, so the task needs no delay. The queue copies the whole block (204 bytes). The Wi-Fi task thus gets its own copy, and the filter task can overwrite `accIn` and `accOut` at once.

Before I added Wi-Fi, the filter task printed each block over the UART. `plot_acc.py` recorded 1050 samples (21 s) with no index gaps, and the board output matched `lfilter()` within 2.6×10^{-4} mg (Figure 4).

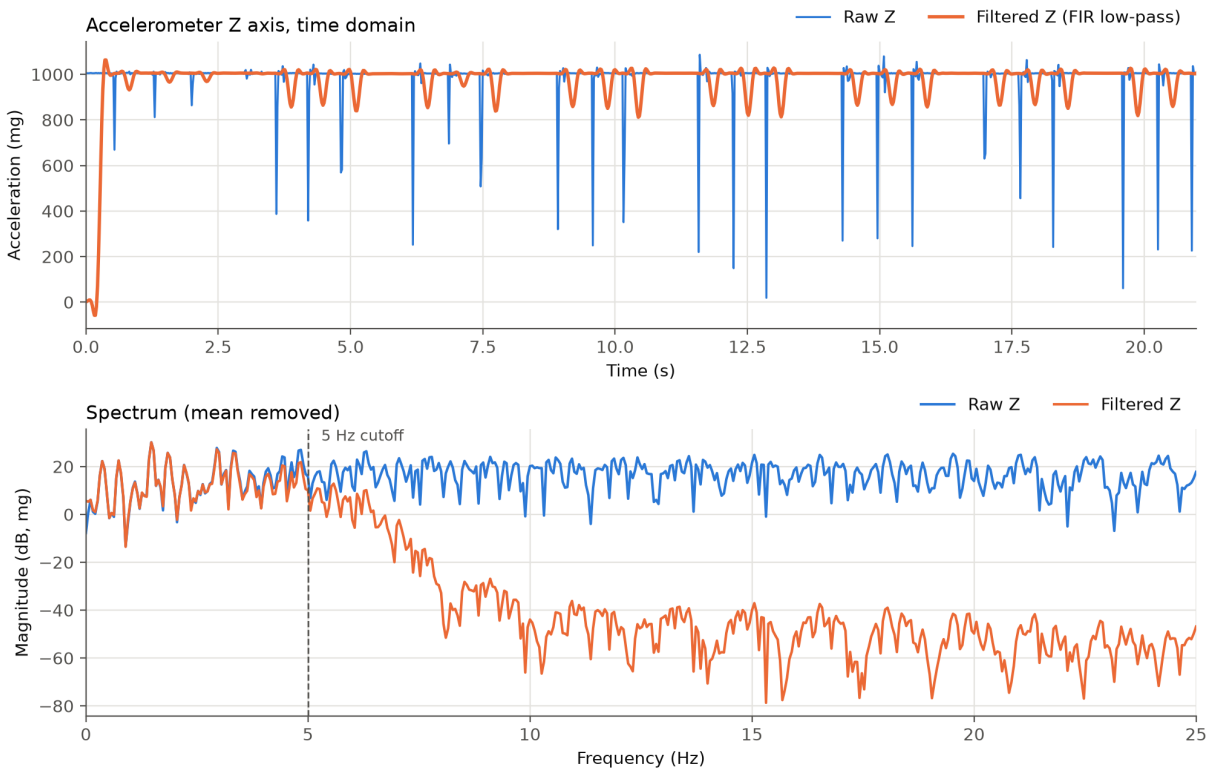


Figure 4: RTOS version with UART output, before Wi-Fi: 21 s with repeated taps.

5.5 Task 3: Wi-Fi

The Wi-Fi task uses the ES-WiFi driver from the STM32CubeL4 `WiFi_Client_Server` example, as in lab 2. The driver talks to the ISM43362 module over SPI3. It needs two interrupts: SPI3 and EXTI1, the data-ready line of the module. The driver sets SPI3 up with its own handle `hspi`, and it enables both interrupts itself. In CubeMX I enabled SPI3 only to get the HAL SPI driver into the build. I left the two interrupts off in CubeMX. Otherwise CubeMX generates an `SPI3_IRQHandler()` for its own handle `hspi3`, and the transfers of the driver never complete. My handlers in `stm32l4xx_it.c` call `HAL_SPI_IRQHandler(&hspi)` and `HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_1)`. `HAL_GPIO_EXTI_Callback()` passes pin 1 to `SPI_WIFI_ISR()`.

The driver runs in its non-RTOS mode. It waits for the SPI flags in a busy loop. This is acceptable because the Wi-Fi task has the lowest of the three priorities. The sensor and filter tasks preempt it at any time, and only the idle task loses CPU time.

At start, the task joins the access point, opens a TCP connection to the PC (port 8002) and sends one test line. Each step prints its result, and a failure stops the sequence. Then the task starts the sample timer. In the loop, the task takes one block from queue B, formats the 25 lines `n,raw,filtered` into one buffer and sends it with one `WIFI_SendData()` call.

```
osTimerStart(sampleTimerHandle, 20);
for(;;)
{
  AccDataBlock local_block;
  if (osMessageQueueGet(accDataQueueHandle, &local_block, NULL, osWaitForever)
      != osOK) {
    droppedBlocks++;
    continue;
  }
  size_t len = 0;
  for (int i = 0; i < ACC_BLOCK_SIZE; i++) {
    len += snprintf(buffer + len, sizeof(buffer) - len, "%lu,%f,%f\n",
                    local_block.n0 + i, local_block.raw[i], local_block.filt[i])
                        ;
  }
  if (WIFI_SendData(0, (const uint8_t*)buffer, len, &sent, 1000) !=
      WIFI_STATUS_OK) {
    sendError++;
  } else if (sent != len) {
    sendError++;
  }
  /* every 20 blocks: send "# dropped=<samples> blocks=<blocks>" */
}
```

Listing 6: Wi-Fi task loop (error prints shortened, drop report left out).

A block is 630 to 725 bytes. The driver cuts any send above 1200 bytes without an error, so the task compares `sent` with `len`. One send per block costs one Wi-Fi command every 0.5 s.

5.6 PC side

`wifi_plot.py` is a TCP server on port 8002. A thread receives the data and splits it into lines. TCP is a byte stream, so one receive can hold half a line or many lines. The plot shows the last 10 s of raw and filtered Z. The time axis is the board's sample index divided by 50 Hz, not the arrival time, because the samples arrive in bursts of 25. The title shows the number of samples, the number of index gaps and the number of bad lines. Lines that start with `#` go to the terminal. At the end, the program saves a CSV file in the format of `plot_acc.py`.

I tested the receiver on the PC first. A test client replayed a recorded CSV in blocks, with a block split in the middle of a line, a missing block, a bad line and a comment line. The receiver

joined the split line, reported one gap, counted one bad line and printed the comment.

5.7 Results

The board joined the access point and got the address 192.168.50.102 in the same network as the PC. Before I moved the timer start, the sensor and filter tasks ran while the Wi-Fi task connected:

```
BEGIN WIFI
WIFI_Init OK
Connecting to SSID: <lab network>
# Dropped Samples: 0
WIFI_Connect OK
Opening TCP client connection to 192.168.50.178:8002
# Dropped Samples: 0
WIFI_OpenClientConnection OK
Sending data...
WIFI_SendData OK, sent 30 bytes
```

Listing 7: Serial output during the Wi-Fi start (timer started at boot, filter task printing the drop counter, network name removed).

`WIFI_Connect()` takes several seconds. During this time queue A lost no sample. The sensor and filter tasks preempted the busy-waiting Wi-Fi task, as the priorities intend.

Figure 5 shows the final version, recorded on the PC from the Wi-Fi stream. It has 1725 samples (34.5 s) with no index gap and no bad line. The filtered column agrees with `lfilter()` on the received raw column within 2.3×10^{-4} mg. The filter on the board and the transfer over Wi-Fi are therefore both correct.

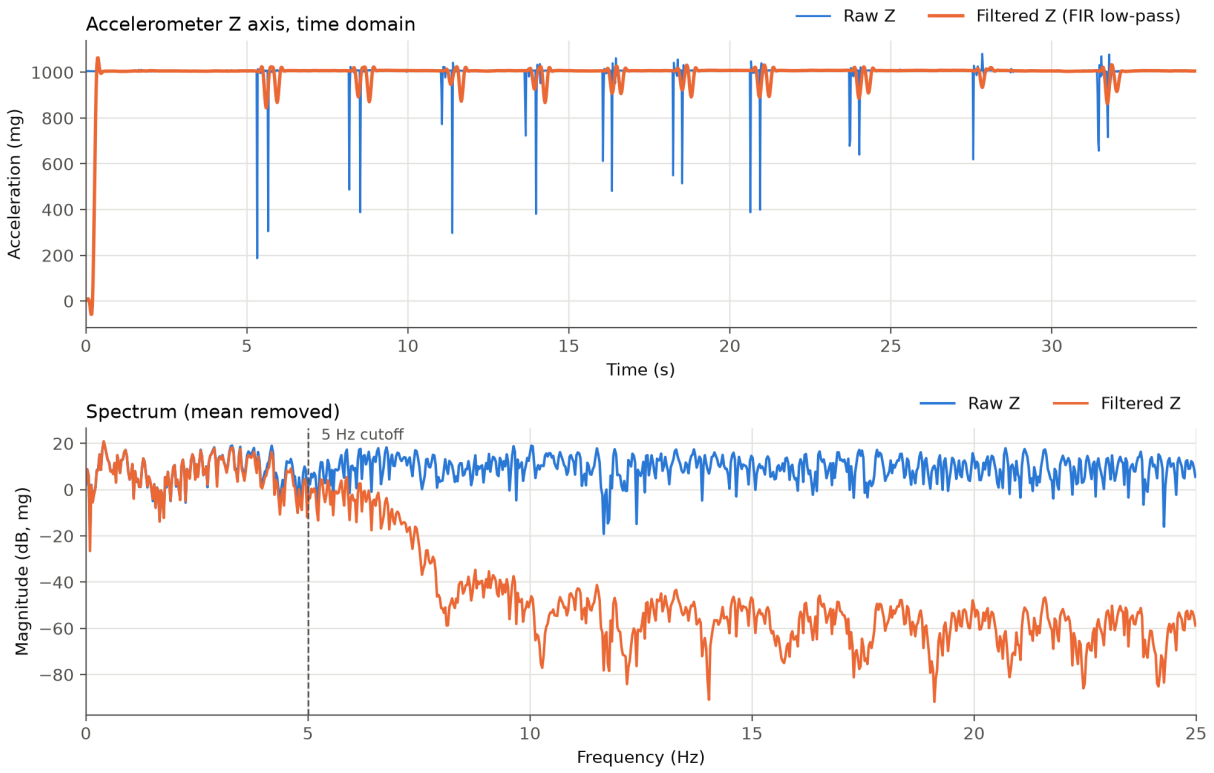


Figure 5: Option problem: raw and filtered Z received over Wi-Fi, with taps every few seconds.

5.8 Discussion

Start-up drops. In the first Wi-Fi version, the timer started at boot. Queue B holds four blocks, so it was full 2 s after boot, while the Wi-Fi task was still connecting. The filter task dropped the next 11 blocks. The PC received samples 0 to 99, then a gap to 375, and the board reported `blocks=11`. $11 \times 25 = 275 = 375 - 100$, so the counter on the board and the gap on the PC agree exactly. Now the Wi-Fi task starts the timer after the connection works, and the stream starts at $n = 0$ with no gap.

Send errors that lose no data. In the same run, 18 sends reported a failure. The driver prints `Send Data command failed` (`es_wifi.c`, line 2311) when the reply of the module after the data does not contain `OK`. But the PC received every block after the start-up gap, once and complete. The error is in the acknowledgement, not in the data. I checked one possible cause: the driver pads odd-length sends with one byte. 79 of the 129 blocks had an odd length, but only 18 sends failed, so the length is not the cause. The cause is still open. The sequence check on the PC is the evidence that no data was lost. A design that needs every block must not resend on this error, because the PC already has the data. It needs a sequence number in each block, as here, and the PC must drop duplicates.

Latency. A sample reaches the PC after up to 0.5 s for the block, plus the group delay of 280 ms, plus the Wi-Fi transfer. The block size sets the trade-off: smaller blocks reduce the delay, but they need more Wi-Fi commands per second. Lab 2 showed that the send rate of this module is limited.

Why the queues, and not shared variables. Without queue B, the Wi-Fi task would read `accIn` and `accOut` while the filter task overwrites them, and lines from two blocks would mix. A draft of the Wi-Fi loop had this bug, and a code review found it before I flashed it. The queue copies the data, so each task owns its own buffer and needs no mutex.

6 Problems Encountered

Table 4 lists the problems in the order they occurred. The rows “Found before flashing” come from code reviews. They never reached the board.

Table 4: Problems, their symptoms and causes.

Problem	Symptom	Cause and fix
Library not found	CMake cannot find the CMSIS-DSP files.	CMake does not expand ~ in paths. I copied the files into `ThirdParty/` and used `${CMAKE_CURRENT_SOURCE_DIR}`.
No float output	`(int)result[0]*1000` printed 3000 for 3.75.	The cast binds before the multiplication. newlib-nano has no `%f`, so I added `-u _printf_float`.
Failure test said "Success"	The log showed Success after the coefficient change.	The log held the old program, which ran until the new one was flashed. The next reset printed 25.16 dB, Failure.
No sensor output	Only the FIR dump appeared.	The sensor code was in `USER CODE 3`, which ends after the closing brace of the loop. It never ran.
1 Mbaud not offered	The VS Code Serial Monitor has no 1 Mbaud setting.	`python -m serial.tools.miniterm COM3 1000000`.
Debugger does not start	"error 138" dialog. The GDB server log says TCP port 61234 is not available.	Windows reserves the ports 61186–61285 for Hyper-V/WSL. `launch.json` sets `serverPort` to 3333.
Timer task priority	No symptom. Found in `FreeRTOSConfig.h`.	The timer task had priority 2, below all tasks. I set it to 48.
Found before flashing	Queue item of 4 bytes from an `int16_t`. `osDelay(1000)` in the filter loop. `osMessageQueueGet()` with a `NULL` buffer.	Wrong data, 49 of 50 samples dropped, and a loop that never waits: CMSIS returns `osErrorParameter` at once for a `NULL` buffer. All three fixed.
Wi-Fi driver does not build	`stm32l4xx_hal_spi.h` missing.	The HAL SPI driver was not in the project. I enabled SPI3 in CubeMX, with its interrupts off.
Gap at start	PC: samples 0–99, then 375. Board: `blocks=11`.	Queue B was full while Wi-Fi connected. The timer now starts after the connection.
Send errors	`Send Data command failed` on 18 sends.	The data arrived anyway (no gap). The cause is open, see the discussion.

7 Conclusion

The CMSIS-DSP FIR filter runs on the Cortex-M4F with the hardware FPU. The known-signal test gives an SNR of 142.6 dB against the MATLAB reference. A deliberate error of one coefficient drops it to 25.2 dB, as a hand calculation predicts. A 29-tap, 5 Hz low-pass filter then smooths the Z acceleration at 50 Hz. Its output agrees with SciPy within 2.3×10^{-4} mg, across all block boundaries.

For option problem 1, three CMSIS-RTOS2 tasks share the work. A timer triggers the sensor task at exactly 50 Hz without a lost trigger. Queues carry the data to the filter task and the Wi-Fi task, and counters on both queues show every lost item. The PC receives and plots the raw and the filtered signal with no gap, and the received data agrees with SciPy.

The main lesson is to make every loss visible. Thread flags, full queues and Wi-Fi sends can all fail without an error. Counters on the board, together with the sample index on the PC, showed the 11 lost blocks at start-up and proved that the 18 send errors lost no data.